

CLR and .NET Fundamentals (DAY 1 Part 2)

1. Common Language Runtime (CLR)

The **CLR (Common Language Runtime)** is the core execution engine of the .NET platform. It is fully responsible for the application lifecycle **from start to termination**.

The CLR handles:

- Loading classes into memory
 - Converting IL to native code
 - Memory management
 - Exception handling
 - Thread management
 - Security checks
-

2. Class Loader

The **Class Loader** is a CLR component responsible for:

- Loading classes from the hard disk into memory
- Loading classes **only when needed** (lazy loading)

This improves:

- Performance
 - Memory usage
-

3. JIT Compiler (MSIL → Native)

When a C# application is compiled:

- The output is **IL (Intermediate Language)**, not machine code

The **JIT Compiler (Just-In-Time)**:

- Converts IL to **native machine code**
- Does this conversion **at runtime**

- Generates machine code specific to the current OS and CPU

C# Code → IL → JIT → Native Code

4. Garbage Collector (GC)

The **Garbage Collector** is responsible for **automatic memory management**.

Its tasks:

- Tracks allocated memory
- Detects unused objects
- Frees memory automatically

This is part of the CLR and removes the need for manual memory deallocation.

Comparison with C/C++

- In C/C++: Developer manages memory manually (pointers)
 - In C#: CLR manages memory automatically
-

5. Managed vs Unmanaged Code

Managed Code

- Executed under CLR supervision
- Memory managed automatically
- Garbage Collector tracks objects
- Safer and more stable

Unmanaged Code

- CLR is NOT responsible
- Developer manages memory manually
- Uses pointers
- Examples:
 - C/C++
 - Low-level system libraries

C# can interact with unmanaged code using **Interop / Marshalling** when needed.

6. Debug Engine

The **Debug Engine** helps developers:

- Debug applications while running
- Trace variables and objects
- Inspect memory and execution flow

This enables:

- Breakpoints
 - Step-by-step execution
 - Runtime inspection
-

7. Security Engine

The **Security Engine** enforces:

- Runtime security rules
- Code access permissions
- Safe execution boundaries

This protects:

- The system
 - The application
 - Other running applications
-

8. Type Checker (Strongly Typed Language)

C# is a **strongly typed language**.

This means:

- Every variable must have a defined type
- Assignments must match the variable type
- Checked at:
 - Compile time
 - Runtime

Example:

```
int x = 5;           // Valid
string y = x;        // Invalid
```

9. Exception Manager

Runtime errors are inevitable.

The **Exception Manager** is responsible for:

- Detecting runtime errors
- Preventing application crashes
- Allowing controlled error handling

In **C#**:

- Exceptions are treated as objects
 - Errors can be handled using `try / catch` blocks
-

10. Thread Support

The **CLR** supports multithreading, allowing:

- Multiple execution paths
- Parallel operations
- Better performance on multi-core systems

Threads enable applications to:

- Perform background tasks
 - Keep the UI responsive
 - Handle concurrent operations
-

11. Base Class Library (BCL)

All previous features rely on the **Base Class Library (BCL)**.

The BCL provides:

- Core data types
- Collections
- File handling
- Input/Output operations
- Threading support
- Exception classes

Languages such as **C#** and **VB.NET** rely on the same BCL.

12. CTS – Common Type System

Different .NET languages use different syntax but share the same underlying types.

Example

C# Type	CLR Type
int	Int32
long	Int64
string	String

This consistency is enabled by the **Common Type System (CTS)**.

13. IL Naming vs Language Naming

Each data type has:

- A **language name** (C# syntax)
- A **CLR name**

Both names refer to the same underlying type in **IL (Intermediate Language)**.

14. Revision: .NET Framework History (Intro)

The **.NET Framework** evolved through multiple versions:

- Introduced with **Visual Studio**
- Multiple framework versions released over time

- Each version added:
 - New APIs
 - Language features
 - Performance improvements

This evolution later led to:

- **.NET Core**
 - **Modern .NET** (5, 6, 7, 8+)
-

Summary (Pages 1–10)

- The **CLR** is the heart of the .NET platform
 - **IL** is converted to native code at runtime
 - Memory is managed automatically using the **Garbage Collector**
 - **C#** is a strongly typed language
 - Exceptions and threads are managed by the **CLR**
 - **CTS** unifies all .NET languages
 - **BCL** provides shared core functionality
-

15. Namespaces (Logical Organization)

A **namespace** is a logical container used to organize classes.

Key points:

- Namespaces are **logical**, not physical
- A single file can contain multiple classes
- Classes are always associated with a namespace

Example:

```
namespace DayOne
{
    class Program { }
}
```

Namespace Example Explanation

This means:

- `Program` belongs to the namespace `DayOne`
- Full name: `DayOne.Program`

Namespaces help to:

- Organize large projects
 - Avoid name conflicts
 - Improve readability
-

16. Physical vs Logical Structure

Physical Structure

- Files
- Folders
- DLLs

Logical Structure

- Namespaces
- Class grouping
- Business separation

Note: A DLL may contain:

- Multiple namespaces
 - Many classes
-

17. EXE vs DLL Projects

EXE Project

- Contains `Main()` method
- Entry point for execution
- Can run directly

DLL Project

- Does NOT contain `Main()`
- Used as a library
- Cannot run alone

When building:

- EXE → `.exe`
 - Library → `.dll`
-

18. Using Namespaces (using Keyword)

To use classes from another namespace:

```
using System;
```

This allows direct access to:

```
Console.WriteLine();
```

Without using:

```
System.Console.WriteLine();
```

19. Adding References & Dependencies

If a class is in another DLL:

- Add reference to the project
- Visual Studio handles linking

Types of references:

1. Framework libraries
 2. Project references
 3. External DLLs
-

20. Data Types Overview

C# data types are categorized into:

Value Types

- `int`
- `float`
- `double`
- `decimal`
- `struct`
- `enum`

Stored directly in memory.

Reference Types

- `class`
- `string`
- `array`
- `object`

Store references to memory locations.

21. Creating Custom Data Types

There are three main ways:

1. `class` → Reference Type
 2. `struct` → Value Type
 3. `enum` → Value Type
-

22. Numeric Types & Precision

Floating Point Types

- `float`
- `double`

Used for scientific calculations.

Decimal

- High precision
- Used for financial calculations

Precision order:

```
decimal > double > float
```

23. Default Values

Each data type has a default value:

- `int` → `0`
- `bool` → `false`
- `string` → `null`

Variables must be assigned before use.

You can assign default explicitly:

```
int x = default;
```

24. Printing Variables

Old Style

```
Console.WriteLine("x = " + x);
```

Format Specifiers

```
Console.WriteLine("x = {0}, y = {1}", x, y);
```

String Interpolation (C# 6+)

```
Console.WriteLine($"x = {x}, y = {y}");
```

Best practice: String Interpolation

25. Reading Input

```
string input = Console.ReadLine();
```

Console input is always read as string.

26. Parsing Input

To convert string to int:

```
int x = int.Parse(input);
```

If input is invalid → **Runtime Exception**.

Safer alternative:

```
int.TryParse(input, out int result);
```

27. Functions & Parameters

Passing by Value

- Default behavior
- Copy of data is passed

Example:

```
void Swap(int a, int b) { }
```

28. Passing by Reference (ref)

- Passes the actual variable
- Changes affect original value

```
void Swap(ref int a, ref int b) { }
```

29. Output Parameters (out)

Used when:

- Function needs to return multiple values

```
int Divide(int a, int b, out int remainder)
{
    remainder = a % b;
    return a / b;
}
```

Caller must initialize out inside the function.

30. ref vs out

Feature	ref	out
Must initialize before call	Yes	No
Used for	Modify existing value	Return extra data

31. Runtime Errors & Exceptions

- Parsing invalid input causes exceptions
 - Division by zero causes exceptions
 - Handled using `try / catch`
-

Final Summary (Pages 10–End)

- Namespaces organize code logically
- DLLs are reusable libraries
- Value vs Reference types differ in memory behavior
- Decimal is best for money

- String interpolation is best for output
- `ref` and `out` control parameter passing
- Proper exception handling is critical